ARCHITECTURE BLUEPRINT

aspgen

Gatewayed generation for scalable .NET systems

Document	Audience	Status
Architecture Blueprint	Engineering teams and maintainers	First release baseline

THESIS: aspgen is a controlled gateway: one command surface composes a project profile, records intent in a manifest, and emits only the layers that the selected architecture can own.

This blueprint explains the theory behind the generator, the boundaries it protects, and three end-to-end paths from command to running application.

Prepared 03 August 2026 • Based on the Architecture Theta ruleset

1. Executive intent

The generator is designed for teams that want speed without surrendering architectural ownership. It treats scaffolding as a bounded design activity: a profile selects the architectural grammar, a manifest records the decision, and incremental commands extend the same grammar without regenerating the whole solution.

DECISION: The generator should optimize for safe composition, not maximum file output. A generated file that has no owning project is a defect, even if its template rendered successfully.

- Clean Architecture protects dependency direction: Domain <- Application <- Infrastructure <- WebApi.
- Vertical slices keep a feature's handler, validator, endpoint, and DTOs close together.
- DDD makes contexts and aggregate roots explicit where business invariants justify the cost.
- Prism modules keep desktop UI composition local, discoverable, and testable.
- The simple profile is intentionally smaller: direct EF Core CRUD for Rails-like throughput.

2. The gatewayed architecture

A gatewayed generator is neither a passive template copier nor a full application framework. It is a policy boundary. Every command passes through four decisions: target application, backend profile, UI profile, and incremental state.

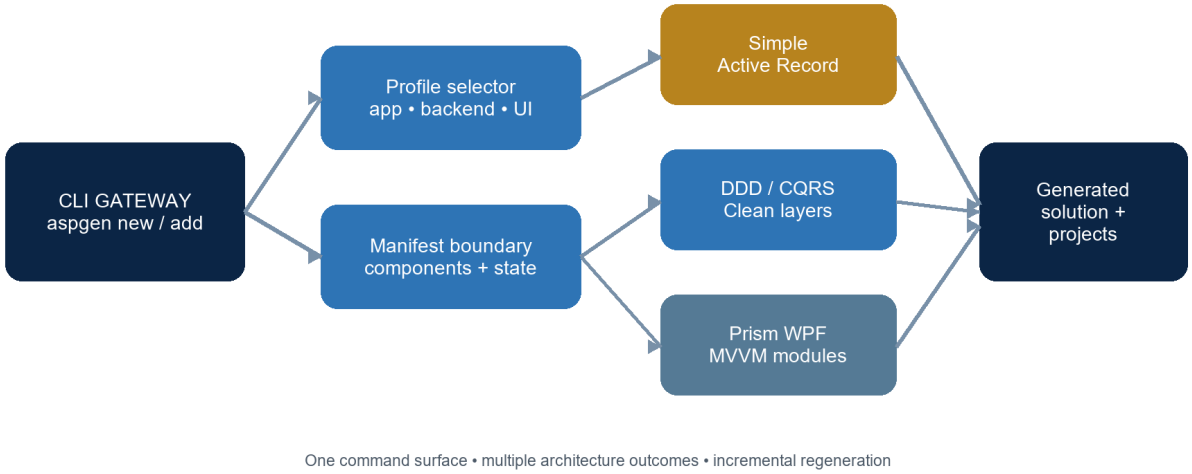

Figure 1. The command gateway turns profile intent into an owned solution graph.

Gateway decision	Question	Result
Application	What host is being built?	webapi, wpf, blazor, or fullstack
Backend	How much architecture is justified?	simple Active Record or DDD/CQRS
UI	Which composition model owns screens?	Prism/Dryloc modules and MVVM
State	What already exists?	.aspgen/manifest.json and markers

3. Patterns and theory

The generator combines patterns that solve different failure modes. The important design move is not to treat them as interchangeable: each pattern has a territory, and the gateway decides when that territory exists.

Pattern / theory	Problem addressed	Generator expression
Clean Architecture	Framework and persistence details leaking into policy	Project references point inward; Domain has no outward dependency.
DDD bounded context	A large model becoming an undifferentiated vocabulary	Contexts scope aggregates, repositories, services, and events.
Aggregate root	Invalid state being changed from arbitrary callers	Commands enter through the root and domain guards protect invariants.
CQRS + vertical slice	Features becoming cross-layer scavenger hunts	Each slice owns command/query, handler, validator, and endpoint.

Pattern / theory	Problem addressed	Generator expression
Prism MVVM	Desktop views knowing application orchestration	Views bind to ViewModels; modules register navigation and regions.
Active Record	Overengineering a small CRUD surface	Simple models and direct EF endpoints reduce ceremony by design.
Progressive delivery	Regeneration destroying local work	Manifest-aware add commands mutate only the requested capability.

RULE OF THUMB Use DDD when invariants and language boundaries are the product. Use the simple profile when CRUD throughput and low ceremony are the product. Do not use complexity as a substitute for a domain decision.

4. Blueprint of a generated solution

Capability	Owned by	Typical generated artifacts
Domain	Domain project	Entities, aggregate behavior, guards, value objects
Application	Application project	Commands, queries, handlers, validators, Result<T>
Infrastructure	Infrastructure project	EF Core context, repositories, audit interceptor, SQLite by default / PostgreSQL opt-in
HTTP host	WebApi project	Minimal API endpoints, OpenAPI, Scalar, health checks
Desktop UI	Desktop project	Prism modules, views, ViewModels, typed events, API stores
Manifest	.aspgen directory	Project identity, components, contexts, backend and seed profile

The solution file lists projects; SDK-style project files automatically include source and XAML below those project roots. The generator therefore treats project ownership—not manual file enumeration—as the visibility invariant.

5. Three examples

Example A — Simple fullstack with dummy data

Use this path for a classic CRUD product, internal tool, or prototype where the shortest safe path to a working screen matters more than domain ceremony.

aspgen new Library --app fullstack --simple --seed:dummy --theme:wpfui aspgen add entity Book title:string author:string pages:int published:date available:bool --project Library

The gateway emits one Web API project and one Desktop project. The entity becomes an EF Core model, direct CRUD endpoints, a WPF module, and three deterministic sample records created at API startup.

Layer	Output
WebApi	Models/Book.cs, Data/AppDbContext.cs, Features/Book/BookEndpoints.cs
Desktop	Modules/Book with list/edit form and API-backed store
Seed	Data/DatabaseSeeder.cs with typed sample values

Example B — DDD/CQRS fullstack

Use this path when the entity carries business meaning, invariants, and a lifecycle that should remain explicit. The backend is split into Domain, Application, Infrastructure, and WebApi projects.

aspgen new Commerce --app fullstack --backend:ddd --seed dummy aspgen add entity Order customerName:string total:decimal submitted:datetime paid:bool --project Commerce

The entity command produces a repository contract, EF repository, DbSet registration, separate CQRS commands and queries, FluentValidation, Minimal API endpoints, a WPF module, and startup seed data. Handler discovery registers IHandler<TRequest,TResponse> implementations automatically.

DEPENDENCY TEST A generated DDD project is healthy only when Domain compiles without Application or Infrastructure references, and the WebApi host composes the layers at the edge.

Example C — Add UI to an existing Web API

Use incremental generation when the server exists first or when a team wants to introduce desktop workflows without regenerating backend work.

aspgen new Operations --app webapi --backend ddd aspgen add entity Person name:string active:bool --project Operations aspgen add ui --theme:wpfui --project

Operations aspgen add module Reports --project Operations

The UI command renders the Desktop project, rewrites the solution membership, and preserves the existing Web API project graph. The module command then adds a Prism module and registers it in App.xaml.cs. No unrelated layers are regenerated.

6. Runtime flow

The same architectural idea appears at runtime: the request crosses a narrow gateway, enters a local slice, applies a domain rule, and reaches persistence only through an owned abstraction.

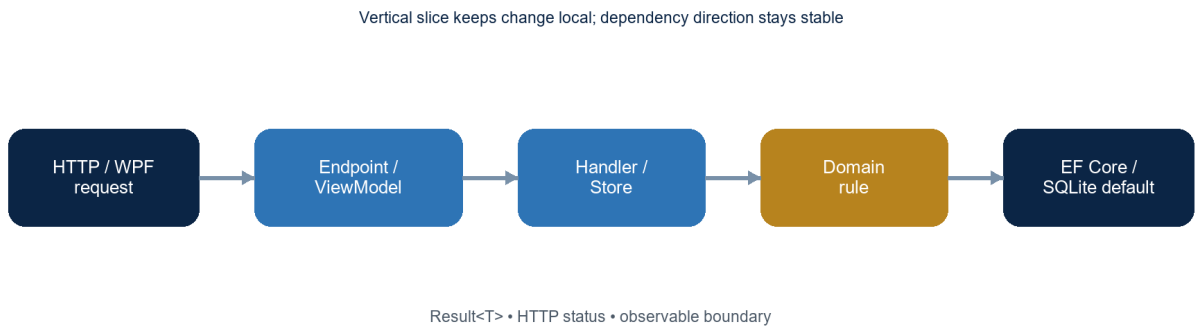

Figure 2. Runtime flow from UI or HTTP boundary to persistence and back.

Map transport input into a request record or ViewModel form model.

Validate at the application boundary; keep endpoint code thin.

Execute a handler or API store operation with async database/HTTP calls.

Enforce invariants in the aggregate or domain entity when DDD is selected.

Return Result<T>, a typed response, or an appropriate HTTP status code.

7. Incremental safety model

Incremental generation is a controlled mutation protocol. The manifest answers what the project is; marker comments answer where a new registration belongs; filesystem tests answer whether the output has an owner.

Safety mechanism	Why it exists	Failure prevented
Manifest components	Persist selected profiles and generated capabilities	A DDD command being applied to a simple project
Host markers	Insert registrations deterministically	Duplicate or misplaced endpoint/module registrations
Refuse overwrite	Protect user-owned files by default	Silent loss of local implementation

Safety mechanism	Why it exists	Failure prevented
Project ownership tests	Verify paths against actual solution roots	Orphan Domain files in WPF-only projects
Generated build checks	Compile representative outputs	Stale DI registrations and template drift

QUALITY GATE: A scaffold is not complete when the command exits successfully. It is complete when the generated files have an owning project, the solution includes that project, and the representative output builds.

8. Recommended operating model

Choose the smallest profile that can express the current use case.

Generate the host and backend once; add entities and modules incrementally.

Run migrations and database updates as an explicit deployment step.

Treat --seed:dummy as development-only data; do not enable it for production environments.

Keep custom changes in source files and use exported templates for organization-wide defaults.

Run go test ./..., go vet ./..., template validation, and at least one generated dotnet build before release.

9. Closing theory

The architecture is intentionally asymmetric. Domain code is conservative and inward-facing; WebApi and WPF are composition edges; the generator itself is a policy gateway that keeps those edges aligned. This makes the system scalable in two directions: simple projects can stay simple, while domain-heavy projects can grow without abandoning the same command vocabulary.

NORTH STAR: Generate less, own more. Every emitted file should have a reason, a project, a boundary, and a path back to a tested running behavior.

Appendix — profile quick reference

Command	Best fit	Primary result
new NAME --app webapi	Clean server baseline	Domain/Application/Infrastructure/WebApi
new NAME --simple	Classic CRUD	Single EF Core WebApi project
new NAME --backend ddd	Business-rich domain	DDD + CQRS + SQLite API; PostgreSQL opt-in
new NAME --app wpf	Desktop-first	Prism/Dryloc WPF shell

Command	Best fit	Primary result
add ui	Web API first, desktop later	Desktop project added to solution
add entity	Vertical slice increment	Backend CRUD, WPF module, or both
--seed dummy	Development demo data	Typed deterministic startup records

Reference basis: Architecture Theta in doc/architecture-theta.md; Prism WPF conventions; Clean Architecture, DDD, CQRS, Vertical Slice, MVVM, and Active Record patterns.